

1List of Experiments:

Do the following seven exercises for any two projects given in the list of sample projects or any other Projects:

1. Development of problem statements.
2. Preparation of Software Requirement Specification Document, Design Documents and Testing Phase related documents.
3. Preparation of Software Configuration Management and Risk Management related documents.
4. Study and usage of any Design phase CASE tool.
5. Performing the Design by using any Design phase CASE tools.
6. Develop test cases for unit testing and integration testing.
7. Develop test cases for various white box and black box testing techniques.

Projects: 1

1. Passport automation System

1: DEVELOPMENT OF PROBLEM STATEMENT

1: Development of Problem Statement

Aim

To develop the problem statement for the Passport Automation System.

Problem Statement

The existing passport application process is manual and takes more time. Applicants have to visit the passport office several times for registration, document verification, and passport collection. This leads to delays and paperwork.

The Passport Automation System provides an online platform where users can register, apply for a passport, upload documents, pay fees, and track the application status.

Objectives

- Reduce manual work.
- Provide online passport application.
- Store applicant details securely.
- Enable online status tracking.
- Improve processing speed.

Modules

- User Registration
- User Login
- Passport Application
- Document Upload
- Fee Payment
- Status Tracking
- Admin Verification

Result

The problem statement was successfully developed.

2: Software Requirement Specification (SRS)

Aim

To prepare the Software Requirement Specification document.

Functional Requirements

- User Registration
- User Login
- Apply for Passport
- Upload Documents
- Pay Application Fee
- Verify Documents
- Track Application Status
- Admin Approval

Non-Functional Requirements

- Security
- Reliability
- Performance
- Availability
- Easy to Use

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB
- Hard Disk: 20 GB

Software Requirements

- Windows 10/11
- HTML, CSS
- PHP
- MySQL
- Google Chrome

Result

Software Requirement Specification was prepared successfully.

3: Design Documents

All diagram draw or printout save file with .mdj

Result

Design documents were prepared successfully.

EX 4: Software Configuration Management and Risk Management

Aim

To prepare the **Software Configuration Management (SCM) plan** and **Risk Management document** for the **Passport Automation System**.

Problem Description

The **Passport Automation System** is an online application that automates passport-related activities such as applicant registration, online application submission, document upload, appointment scheduling, application verification, fee processing, and application-status tracking.

Software Configuration Management

Configuration Items

- SRS Document
- Design Documents
- Source Code
- Database
- Test Cases

Version Control Tool

Git

STEP 1: Create a GitHub Account

1. Open a web browser.
2. Go to [GitHub](https://github.com).
3. Click **Sign up**.
4. Register using the **college official email ID**.
5. Create the GitHub account.
6. Verify the college email address.
7. Login to GitHub.

Username

Use the **Roll Number** as the GitHub username if available.

Example:

Username: 23CSE001

STEP 2: Install Git on Windows

1. Open the official Git website.
2. Download **Git for Windows**.

[Download Git for Windows](#)

3. Run the downloaded installer.
4. Click **Next** through the installation options.
5. Keep the default settings
6. Complete the installation.

STEP 3: Verify Git Installation

Open **Command Prompt** or **Git Bash**.

To open Git Bash

Press:

Windows Key → type Git Bash → Enter

Execute:

```
git --version
```

Expected output will be similar to:

```
git version 2.x.x
```

STEP 4: Configure Git

Open **Git Bash** and configure your name.

```
git config --global user.name "Your Name"
```

Example:

```
git config --global user.name "Dr.Yasmeen"
```

Configure the email address associated with GitHub:

```
git config --global user.email "yourcollegeemail@example.com"
```

Verify the configuration:

```
git config --global --list
```

You should see:

```
user.name=Your Name  
user.email=yourcollegeemail@example.com
```

STEP 5: Create the Project Folder on Windows

```
mkdir Passport-Automation-System
```

```
cd Passport-Automation-System
```

```
pwd
```

STEP 6: Initialize the Local Git Repository

Inside the project folder, execute:

```
git init
```

```
git status
```

STEP 7: Create README.md

Create the README file.

You can create it using Notepad, VS Code, or Git Bash.

Using Git Bash:

```
touch README.md
```

```
notepad README.md
```

```
# Passport Automation System
```

```
This project is created for academic demonstration purpose only.
```

```
Software Configuration Management using GitHub.
```

```
Student Name: <Your Name>
```

Roll Number: <Your Roll Number>

Class: <Your Class>

> Save and close the file.

STEP 8: Check Git Status

git status

git add README.md

optional -- **git add .**

git status

STEP 9: Create the First Commit

git commit -m "Initial project documentation"

STEP 10: Rename the Branch to Main

git branch -M main

git branch

Create the GitHub Repository

STEP 10: Connect Local Repository to GitHub

git remote add origin <https://github.com/23CSE001/Passport-Automation-System.git>

git remote -v

STEP 11: Push the First Version to GitHub

git push -u origin main

Add the Project File – code.html

notepad code.html

<!DOCTYPE html>

<html>

<head>

<title>Student Information</title>

```
</head>
```

```
<body>
```

```
<h1>Passport Automation System</h1>
```

```
<h2>Student Information</h2>
```

```
<p>Student Name: Your Name</p>
```

```
<p>Roll Number: Your Roll Number</p>
```

```
<p>Class: Your Class</p>
```

```
</body>
```

```
</html>
```

Save and close the file.

```
git status
```

```
git add code.html
```

View Version History

Simple history

```
git log --oneline
```

Complete graphical history

```
git log --oneline --all --graph
```

optional

```
git config --global credential.helper manager
```

```
git config --global --get credential.helper
```

Risk Management

1. Risk Identification

Risk ID	Risk	Probability	Impact	Risk Level
R1	Loss of applicant data	Medium	High	High
R2	Unauthorized access	Medium	High	High
R3	Server failure	Medium	High	High
R4	Requirements changes	High	Medium	High
R5	Delay in development	Medium	Medium	Medium
R6	Document upload failure	Medium	Medium	Medium
R7	Database corruption	Low	High	Medium
R8	Payment processing failure	Medium	High	High
R9	Integration failure with external services	Medium	High	High
R10	Insufficient testing	Medium	High	High

Result

SCM and Risk Management documents were prepared successfully.

5: Study and Usage of CASE Tool

Aim

To study and use a CASE tool.

CASE Tool

StarUML

Procedure

1. Install StarUML.
2. Create a new project.
3. Draw Use Case Diagram.
4. Draw Class Diagram.
5. Draw Activity Diagram.
6. Save the project.

Advantages

- Easy diagram creation.
- Professional UML diagrams.
- Saves development time.
- Easy to modify designs.

Result

StarUML was used successfully to create UML diagrams.

6: Unit Testing and Integration Testing

Aim

To develop unit and integration test cases.

Unit Test Cases

Test ID	Module	Input	Expected Output
UT01	Login	Valid User	Login Success
UT02	Login	Invalid Password	Error Message
UT03	Registration	Valid Data	Registration Success
UT04	Payment	Valid Payment	Payment Success
UT05	Upload	Valid File	Upload Success

Integration Test Cases

Test ID	Modules	Expected Result
IT01	Registration + Login	Login Successful
IT02	Application + Upload	Documents Uploaded
IT03	Payment + Application	Application Submitted
IT04	Verification + Approval	Passport Approved

Result

Unit and Integration Testing were completed successfully.

7: White Box and Black Box Testing

Aim

To perform White Box and Black Box Testing.

Black Box Testing

Valid Inputs

- Correct Username
- Correct Password
- Valid Application Number

Invalid Inputs

- Empty Username
- Wrong Password
- Invalid Application Number

Boundary Value Analysis

Age	Result
17	Invalid
18	Valid
30	Valid
60	Valid

White Box Testing

Program Logic

If documents are verified
 Approve Passport
Else
 Reject Passport

Statement Coverage

Execute every statement at least once.

Branch Coverage

- Documents Verified = Yes
- Documents Verified = No

Result

White Box and Black Box Testing were performed successfully.

Overall Conclusion

The **Passport Automation System** was successfully analyzed, designed, and tested. The project demonstrates the complete Software Development Life Cycle (SDLC), including requirement analysis, design, configuration management, testing, and risk management. The system simplifies the passport application process by reducing manual work and improving efficiency.













